

Engineering Systems Multiple-Domain Matrix: An Organizing Framework for Modeling Large-Scale Complex Systems

Jason E. Bartolomei,* Daniel E. Hastings, Richard de Neufville, and Donna H. Rhodes

Massachusetts Institute of Technology, 77 Massachusetts Avenue, Cambridge, MA 02139

Received 21 July 2009; Revised 21 October 2010; Accepted 24 February 2011, after one or more revisions

Published online 10 October 2011 in Wiley Online Library (wileyonlinelibrary.com).

DOI 10.1002/sys.20193

ABSTRACT

The scope and complexity of engineered systems are ever-increasing as burgeoning global markets, unprecedented technological capabilities, rising consumer expectations, and ever-changing social requirements present difficult design challenges that often extend beyond the traditional engineering paradigm. These challenges require engineers and technical managers to treat the technological systems as a part of a larger whole. Existing system modeling frameworks are limited in scope for representing the information about engineering systems. This paper presents a conceptual framework and an improved modeling framework for engineering systems. **Its value is that it allows engineers and managers an improved means to visually arrange information and structure discourse in a way that facilitates better systems engineering.** It augments the existing literature by providing a clear and concise framework for an engineering system, and provides a methodology for engineers to tag and organize systems information in ways that allow for better collection, storage, processing, and analysis of systems engineering data. © 2011 Wiley Periodicals, Inc. *Syst Eng* 15: 41–61, 2012

Key words: engineering systems; systems theory; social and technological systems; Design Structure Matrix (DSM); coupled DSM; Engineering Systems Matrix (ESM); Engineering Systems Multiple-Domain Matrix (ES-MDM); Multiple-Domain Matrix (MDM); knowledge basis

1. INTRODUCTION

Engineering systems (ES) is a field that seeks solutions to important, multifaceted sociotechnical problems [ESD, 2008]. ES has emerged in response to the growing complexity of human technological endeavors and the insufficiency of extant theoretical knowledge and understanding to guide engineers, managers, and policy makers responsible for the

design and management of large-scale complex systems. It encompasses activities [CESUN, 2010]:

- Systems Engineering
- Technology and Policy
- Engineering Management, Innovation, Entrepreneurship
- System and Decision Analysis, Operations Research
- Manufacturing, Product Development, Industrial Engineering.

The problems ES seeks to tackle are not new, as several disciplines have researched, developed, and applied methods and tools for decades. However, Rouse [2007] explains that large-scale complex systems require a broad “enterprise per-

*Author to whom all correspondence should be addressed at: 10147 Forney Loop, Ft. Belvoir, VA 22060 (e-mail: jason.bartolomei@mit.edu; hastings@mit.edu; ardent@mit.edu; rhodes@mit.edu).

spective” that integrates the behavioral, social, and life sciences, as well as management. ES seeks to integrate these disciplines to discover the fundamental principles and properties of systems aimed at fulfilling important functions in society (large-scale), characterized by a high degree of technical complexity, social intricacy, and elaborate processes (complex). Examples of these large-scale complex systems, or *engineering systems*, include critical infrastructure, health-care delivery systems, and manufacturing systems [ESD, 2008].

Rouse [2007] presents several fundamental areas of research for large-scale complex systems in biological, engineering systems, and natural systems domains. These include:

- The full design objectives for such systems
- Approaches to architecting and modeling systems relative to these objectives
- A methods and tools for model development and use
- Means for evaluation and experimentation with models and real systems
- Approaches, technologies and tools to enable decision support for those who invest in, develop, operate, and use complex systems.

To advance scholarship in these fundamental areas of research, scholars must address underlying philosophical questions related to the ontology (the study of the nature) and epistemology (the study of how we learn and know) of engineering systems. This includes foundational research that explores questions like:

- What do we mean by Engineering Systems?
- What information is required to describe an Engineering System?
- Where does this information exist?
- How do we observe these systems?
- Do we have the tools to develop theories about them?
- What are the ontological or epistemological challenges created by integrating social sciences, engineering, and management?

This paper examines these questions by first reviewing the systems literature to identify existing theoretical gaps. Next, we present a first-order ontological model of an engineering system. Using this conceptualization, we present an organizing framework for modeling these systems, called the *Engineering Systems Multiple-Domain Matrix*. We explore what information is required to describe an engineering system, where this information resides, a technique to elicit this information, and methods for putting this information to use.

For practitioners, this paper provides an improved means to construct a knowledge basis that facilitates better systems engineering. In addition, it also presents a methodology for engineers to tag and organize systems information in ways that allow for better collection, storage, processing, and analysis.

2. LITERATURE REVIEW

Systems engineering is one of several bodies of knowledge to have emerged using a systems approach for organizing and

interpreting the world [Brill, 1999]. The systems view gives a distinct view of humans and nature [Laszlo, 1972]. It contrasts with a more traditional, reductionist view. Rather than disaggregate systems into simpler and simpler parts, the systems approach embraces a holistic view of the world [Popper, 1961, 1972; M’Pherson, 1974]. A classic definition of a “system” is the integration of a set of elements into an orderly whole that functions as an organic unity [Simon, 1962; Marchal, 1975; Rescher, 1979]. The organic unity displays holistic properties greater than the sum of the parts, which are defined as “emergent” properties [Simon, 1962; Bertalanffy, 1968; M’Pherson, 1974; Moses, 2004].

A holistic view has been useful to understand various phenomena; including efforts to understand the social [Parsons, 1964; Miller, 1978], biological [Bertalanffy, 1968; Miller, 1978; Kitano, 2002], economic [Boulding, 1956; Forrester, 1961], ecological [Pielou, 1969; Graedel and Allenby, 2003], historical [Callon, 1990; Hughes, 1983, 1987, 1990, 1998], political [Quade and Boucher, 1968; Vickers 1983; Allison and Zelikow, 1999], organizational [Beer, 1967; Ackoff, 1973; Senge, 2006], and technical [Weiner, 1948; Buede, 2000; Sage, 1992, Sage and Armstrong, 2000]. Consequently, several systems subfields have emerged spanning disciplinary boundaries as scholars have found that understanding different types of problems require diverse knowledge.

Systems engineering does not seem to fit nicely in any of the existing typologies. This is a theoretical gap in the systems literature. Take, for instance, a community hospital. Most would agree that a hospital is a complex system in that it consists of many interacting parts that exhibit well-defined, albeit often poorly understood, behaviors. However, if one considers the hospital’s organizational components, infrastructure, technological devices, and the processes as constituent parts of the system, it becomes difficult to classify within the existing typologies. Outside the systems engineering literature, engineering systems have been described, and theoretical roots for the study of these systems have been planted in the social-psychology and history of technology literature. These include sociotechnical system theory [Trist and Bamforth, 1951] and large technological systems [Hughes, 1983].

2.1. Sociotechnical Systems Theory

Trist and Bamforth [1951; Trist, 1953] developed a sociotechnical systems framework for understanding organizational behavior that moved beyond simply observing social interactions, to include explicit consideration of work-tasks and technical systems [Badham et al., 2000]. Their work began a subdiscipline within industrial psychology and has since exported several concepts and theories to other disciplines including organization studies, human factors engineering, and management.

Emery [1993] provides an overview of the basic theoretical concepts underlying sociotechnical systems (STSs). He sees them as open systems that interact with an environment. The concept of purpose is particularly important as STSs consist of social/organizational and technical components that interact to achieve the purpose of the system, usually the production of a good or delivery of a service.

Central to STS theory is the development of concepts of work relationships, human task and task interdependencies, and organizational structures that contribute to the production process. STS is devoted to understanding the social psychological factors associated with human-machine interactions (level of automation), team structures, and industrial organizational strategies.

2.2. Large Technological Systems

Hughes developed a comprehensive description for engineering systems he calls Large Technological Systems (LTSs) that build on many concepts from systems theory [Hughes, 1983, 1987, 1990, 1998]. He presents a description of the qualities of LTS, rather than a precise definition. Hughes describes LTS as a seamless web of diverse components that span several domains. LTS components include physical artifacts (technical), legislative artifacts (constraints), organizations (social), and natural resources. Hughes defines LTSs as open systems that interact within an environment. For Hughes, the environment consists of two types of factors, those that are independent and those that depend on the system. An LTS is distinguished from its environment by the limits of control exercised by the system's components. These limits define the system boundary.

Hughes defines LTS as purposeful systems that exist to solve problems or fulfill goals, having mostly to do with "reordering the physical world in ways considered useful or desirable" [Hughes, 1987]. He adds that unlike the "other disciplines of art, architecture, medicine, and play...[LTSs are] usually concerned with the reordering of the physical world to make it more productive of goods and services." He also describes a concept of traceability. Traceability implies that all functioning components of the system "contribute directly or through other components to the common system goal" [Hughes, 1987] or purpose. He pays particular attention to large-scale systems, which he describes as those technological systems that are society-shaping, and he offers several examples of this type of system, such as the U.S. Electric Power Grid.

Hughes discusses the concept of system structure as both the social and technical components of the system display a hierarchical structure. He explains that "the inventors, organizers, and managers of technological systems mostly prefer organizational hierarchy" [Hughes, 1987]. As a result, over time the technical artifact will tend to a hierarchic structure as well. His discussion of hierarchy aligns well with Simon's [1962] discussion of "The Architecture of Complexity."

In addition to defining and understanding the structure of LTS, Hughes places particular emphasis on the dynamic behavior of the system. Through his work, he successfully demonstrates that certain system behaviors can only be understood as sociotechnical phenomena. He cautions researchers to be careful to select the level of analysis, particularly for hierarchic systems, as in the case of "large technological systems there are countless opportunities for isolating subsystems and calling them systems for purposes of comprehensibility and analysis." Although he does not use the word, Hughes describes the concept of nested complexity, or the behavioral complexity exhibited between subsystems,

at the various levels of a hierarchical systems. Hughes warns that the risk of not carefully understanding the appropriate levels of analysis can lead to "only a partial, or even distorted, analysis of system behavior."

Hughes introduces several system behaviors in his discussion of the patterns of the evolution of LTS. He develops concepts and explains system behaviors that involve social and technological interactions, including invention, innovation, development, momentum, and reverse salients and several others. Hughes uses the LTS as a "less elegant but useful" systems approach for understanding the history of technology [Hughes, 1987].

3. CONCEPTUALIZING ENGINEERING SYSTEMS

The accepted description of an engineering system is congruent with LTS and STS theory. Engineering systems are characterized by a high degree of technical complexity, social intricacy, and elaborate processes, aimed at fulfilling important functions in society. Bartolomei [2007] unpacks this description and presents a first-order ontological model, or conceptualization, of an engineering system.

3.1. A Conceptualization for an Engineering System

A conceptualization is an abstract, simplified view of the world that is represented for some purpose. It provides a basis for a body of formally represented knowledge, that is, the objects, concepts, and other entities that are assumed to exist in some area of interest [Genesereth and Nilsson, 1984]. Explicitly or implicitly, every engineering model stems from some conceptualization that describes its concepts, classes of system components, and relationships.

This paper's conceptualization for engineering systems seeks to address a theoretical gap that moves beyond existing descriptions towards an ontological model. It formalizes the identification and definition of the domains common to all engineering projects. Figure 1 represents our view of engineering systems domains, components, and relationships. These domains include the following:

- **Environmental**, the exogenous components that affect or are affected by the engineering system
- **Social**, the social network consisting of the human components and the relationships that holds among them
- **Functional**, including the goals and purposes of the engineering system, as well as its functional architecture
- **Technical**, the physical, nonhuman components of the system to include hardware, infrastructure, software, and information
- **Process**, the processes, subprocesses, and tasks performed within or by the system.

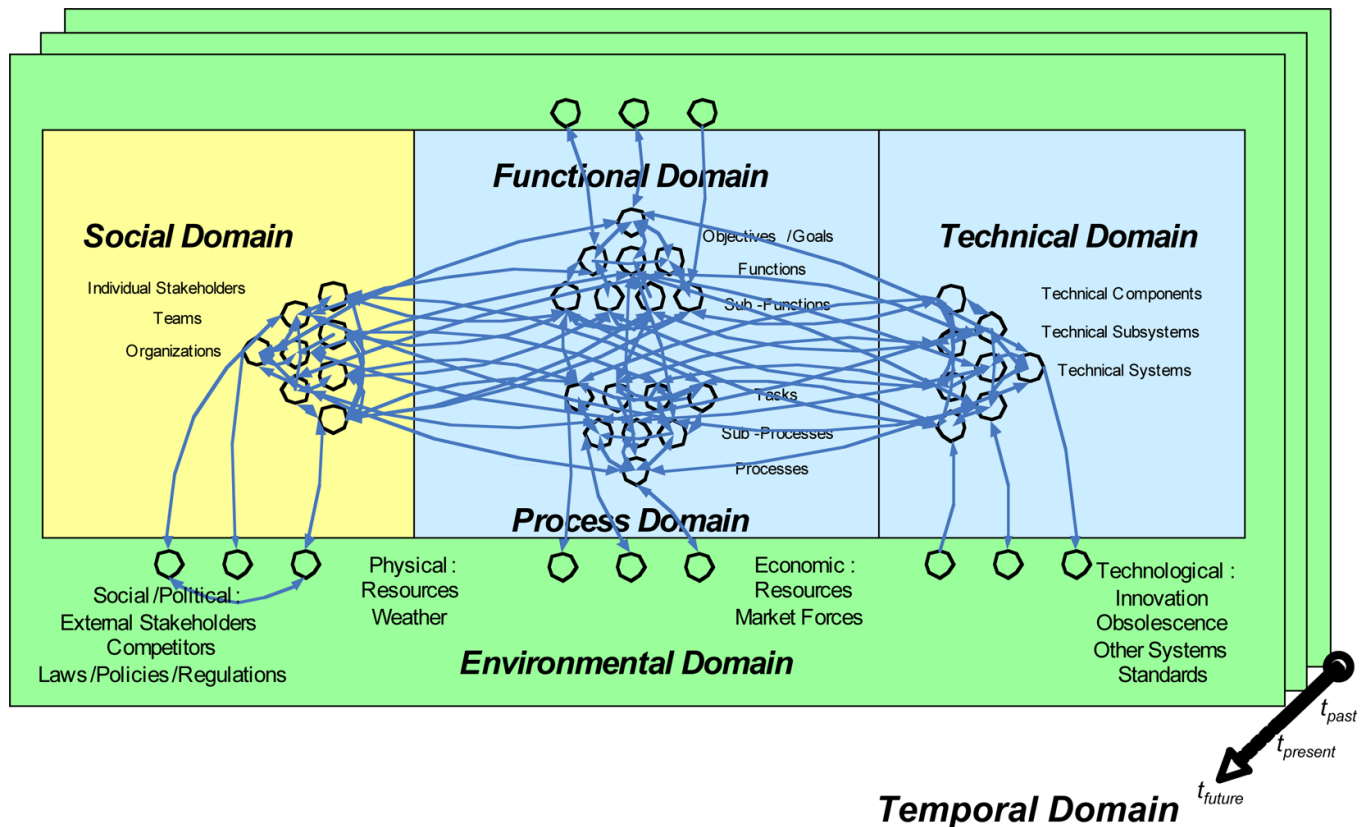


Figure 1. Engineering system conceptual model. [Color figure can be viewed in the online issue, which is available at wileyonlinelibrary.com.]

3.2. Time

Engineering system components and their environment change over time. Components and their properties can change, be added, or be dropped. An engineering system exhibits emergent properties that result from interactions between the social and technical components. The system property “flexibility,” defined as the measure of ease for which a system can change over time, is an emergent property of the system that can only be understood by examining the social and technical domains. To measure flexibility, a system engineer must understand sources of change, how change will affect the system components, and who is responsible for authorizing and managing the change.

3.3. System Boundary

Engineering systems are bounded by their constituent components limits of control and alignment with the system’s purpose. The components of an engineering system constitute a unified whole that interacts with exogenous components as an open system.

An engineering system may have components that no longer contribute to the goal of the system and remain passive. Some components may detract from the goals. For example, a human component may have localized goal(s) that detract from the global goals [Merton, 1957]. Because engineering systems are cybernetic, the system should adjust, to address

dysfunction and bring order to the system, but this does not always happen [Easton, 1965]. Thus some employees may fight a policy, and the system will self-regulate and adjust to bring components into alignment with the goals of the system. It may replace the components, change policies, or make other adjustments. In some cases, if dysfunction is not addressed, the engineering system can cease. Mutations and adaptation are possible as well. Engineering systems are complex adaptive systems.

3.4. Other Properties

Path dependence is another property of engineering systems. The current state of the system depends on previous states. Because engineering systems are socially constructed, one might think of an engineering system as the product of thousands of human decisions over time. Thus, changes of an engineering system can be observed and recorded. Thus, efforts to understand the consequences of human decisions, unexpected events, and other system behaviors require a deep understanding of each of the domains.

Engineering systems can exist at various scales of complexity. Whereas ES as a field focuses attention on “large-scale” complex systems, our conceptualization describes engineering systems at varying degrees of complexity, from the simplest of a single human interacting with an artifact for some purpose (fighter pilot and jet), to transnational suprasys-

tem that includes possibly millions of system components (NATO).

The structure of a technical system often reflects that of its organization [Hughes, 1987]. We thus feel the social domain should be the basis for defining the levels of complexity of an engineering system. Our descriptions of its various levels are then:

- **Individual:** a person interacting with technical components for some purpose (a pilot operating an aircraft)
- **Group:** a collection of individuals interacting with technical components and each other, where their localized contributions support the global goal of the group (a fighter squadron with multiple pilots operating aircraft)
- **Organization:** interacting groups, whose local contributions support the organization's goal (a fighter wing of multiple squadrons)
- **Enterprise:** interacting organizations, whose contributions support the enterprise (the US Air Force, consisting of multiple organizations)
- **Higher-Order:** enterprises, organizations, and groups, where each supports higher-level goals.

Higher-order levels of complexity may exist as some engineering systems have multiple layers of hierarchy, and it may be difficult to delineate between the different levels of complexity. The US Department of Defense (DoD) can be conceptualized as an engineering system as the localized goals of the constituent components (the services and others) exist to support its global goals. Each service can be understood as a system with well-defined system boundaries, and each would probably fall under the enterprise level of com-

plexity. Therefore, the DoD is an example of an engineering system of a higher-order complexity. Other such examples may include NATO, OPEC, and the United Nations. ES is devoted to the study of this type of system.

3.5. Existing Conceptual Models for Engineering Systems

Several conceptual models currently underlie well-established systems modeling frameworks. These include the Design Structure Matrix [Browning, 2001; Eppinger, 2003; Guenov and Barker, 2005; Steward, 1981]; the Domain Mapping Matrix [Danilovic and Browning, 2007]; Axiomatic Design [Suh, 1998, Guenov and Barker, 2005]; Unified Program Planning [Warfield and Hill, 1972]; Quality Functional Deployment/House of Quality [Guina and Praizler, 1993]; CLIOS: Complex, Large-scale, Interconnected, Open, Sociotechnical System [Dodder et al., 2005]; and DoDAF, the DoD Architecture Framework [DoD, 2003]. Bartolomei [2007] examined these frameworks in detail and found that each of them were limited in scope. Some fail to represent important interactions across domains, capture concepts of time, or adequately include both the social and technological aspects of the system.

The DoDAF is one of the most comprehensive systems modeling frameworks. In the mid-1990s the US DoD created the C4ISR architectural framework to integrate various military communication and surveillance weapons systems. Within this framework there were three main views of the architecture, for the Operational (OV), the Systems (SV), and the Technical (TV). Each represents a standardized collection of models using different modeling languages to represent a predefined aspect of the system. Each view provides details

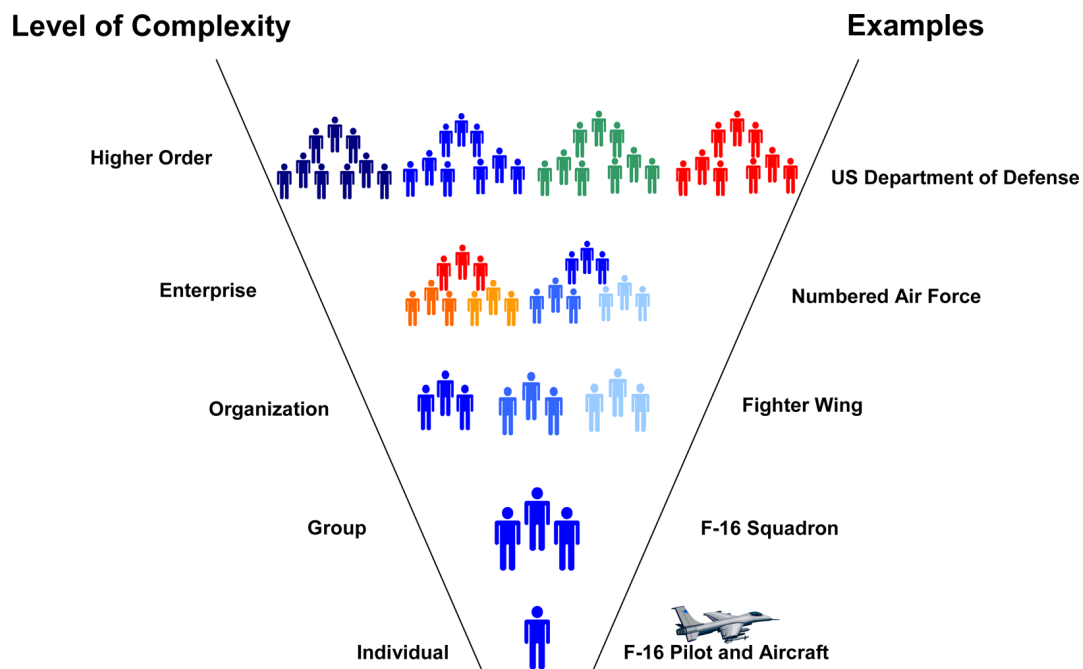


Figure 2. Levels of complexity. [Color figure can be viewed in the online issue, which is available at wileyonlinelibrary.com.]

about the system valuable to the various stakeholders involved in developing or actively managing the system. The views were designed for different user groups surrounding a technological system. **Military planners used the OV to define the concept of operations for a particular weapon system. Sub-views within the OV ranged from simple graphical descriptions of battlespace to high-level network diagrams easy for nonengineers to understand. On the other hand, the SV and TV included far more sophisticated engineering models to be used for technical communication.**

The DoD based the DoDAF, a mandatory work product for every weapon system in the US inventory, on the C4ISR architectural framework. DoDAF Version 1.5 consisted of 26 products representing the operational, systems, and technical views of a system [Cooper et al., 2005; Richards et al., 2006]. Each view is described through a set of “products” that includes diagrams, tables, graphics, and narratives [see DoD, 2003].

There were limitations in implementing the DoDAF. Little documentation was provided on how to construct them. Coupled with a focus on final view outputs in early user training, this led to a work product-centric approach to DoDAF development. As a result, many early DoDAF work products were pictures that were neither internally consistent nor complete in capturing relevant data. Furthermore, the DoDAF provided a discrete picture of each individual view; thus it is impossible to capture the dependencies and parallelisms among activities, processes, and supporting technologies [Richards et al., 2006]. The lack of internal consistency was a problem both procedurally and structurally, in that the information presented in discrete views is not traceable to the other views. For example, OV-4 presented a diagram of the organization surrounding a system; however, there are no views that relate the organizational entities to elements in the technical and systems views or the nature of the relations.

In 2009, DoDAF 2.0 was released to address many of these shortcomings [DoD, 2009]. **An expanded set of viewpoints spanning the social, technological, and environmental domains replaced the architecture views.** A DoDAF Meta Model (DM2) integrated architecture viewpoints, to insure internal consistency and enhance the analytical and evaluative value of the framework. DoDAF 2.0 also captured information on how the systems interact with exogenous systems (**capability viewpoints**) and incorporated the idea of time for a number of viewpoints (i.e., CV-3 Capability Phasing, OV-6b State Transition Description, PV-2 Project Timelines, SvcV-8 Services Evolution Description, SvcV-9 Services Technology & Skills Forecast, StdV-2 Standards Forecast).

Software applications were developed to streamline DoDAF construction. These promote internal consistency across views, improve data management, and some provide basic modeling and simulation capabilities. These advances provide procedural improvements in consistency and efficiency in model building.

DoDAF 2.0 is moving in the right direction. The scope of its underlying ontological model has expanded significantly. However, the means to represent important social, political, and economic considerations remain underrepresented. Future iterations of DoDAF should seek to better capture these considerations, and improve means to capture uncer-

tainty/risk, dynamic complexity, and include better hooks for various quantitative analyses.

4. THE ENGINEERING SYSTEMS MULTIPLE-DOMAIN MATRIX

This paper presents the Engineering Systems Multiple-Domain Matrix (ES-MDM) as an organizing framework for modeling engineering systems. The ES-MDM operationalizes the conceptualization of an engineering system and addresses the limitations of existing frameworks by providing means to arrange information and structure discourse in a way that facilitates better systems engineering.

Visualize the ES-MDM as an adjacency matrix with identical row and column headings (see Fig. 3). The diagonal represents the system components, and the off-diagonal cells represent the relationship between components. The cells along the diagonal represent a graph of a particular class of node. Each node class aligns with one of the engineering systems domains. The off-diagonal blocks of cells represent multi-partite relations that relate various classes of nodes [Steward, 1981; Bartolomei et al., 2007; Danilovic and Browning, 2007; Maurer and Lindeman, 2008].

For those familiar with IEEE 1471, one can interpret the ES-MDM as a collection of models addressing the whole system. Each row/column of the ES-MDM represents a “view,” or a particular aspect or a set of concerns. Moreover, the ES-MDM establishes a basis for view-view interactions, giving a more formalized, theoretically grounded perspective of an engineering system.

The ES-MDM is designed as a rich data structure able to represent the pertinent information for designing and managing an engineering project. The ES-MDM is both a hypergraph and a multigraph. A hypergraph implies the graph contains different classes of nodes and there are interactions between nodes of different types (e.g., people and processes). A multigraph implies that multiple edge-types can exist between nodes. For example, two social actors might have both a financial and a communication relationship between them. (see Fig. 4)

Each node and relation in the system can be described with **attributes**. An example of an object and corresponding attributes are shown in Figure 5. Attributes can be binary, string, numeric, or a mathematical function. Lastly, the ES-MDM represents how the system (nodes, relations, and attributes) changes over time.

The ES-MDM data structure consists of six node classes that correspond to the five engineering systems domains. Below is a short description of each node class, with their interactions within each domain. We use a community hospital to highlight the concepts. Community hospitals display qualities of engineering systems by exhibiting a high degree of technical complexity, social intricacy, and elaborate processes [ESD, 2008].

4.1. System Drivers

Starting in the northwest corner, the Systems Drivers Matrix represents the environmental domain. It comprises the factors

	Environmental Domain	Social Domain	Functional Domain		Technical Domain	Process Domain
	System Drivers	Stakeholders	Objectives	Functions	Objects	Activities
System Drivers	System Drivers X System Drivers	Stakeholders X System Drivers	Objectives X System Drivers	Functions X System Drivers	Objects X System Drivers	Activities X System Drivers
Stakeholders	System Drivers X Stakeholders	Stakeholders X Stakeholders	Objectives X Stakeholders	Functions X Stakeholders	Objects X Stakeholders	Activities X Stakeholders
Objectives	System Drivers X Objectives	Stakeholders X Objectives	Objectives X Objectives	Functions X Objectives	Objects X Objectives	Activities X Objectives
Functions	System Drivers X Functions	Stakeholders X Functions	Objectives X Functions	Functions X Functions	Objects X Functions	Activities X Functions
Objects	System Drivers X Objects	Stakeholders X Objects	Objectives X Objects	Functions X Objects	Objects X Objects	Activities X Objects
Activities	System Drivers X Activities	Stakeholders X Activities	Objectives X Activities	Functions X Activities	Objects X Activities	Activities X Activities

Figure 3. Engineering Systems Multiple-Domain Matrix. [Color figure can be viewed in the online issue, which is available at wileyonlinelibrary.com.]

that act or are acted on by the system [Bunge, 1979]. The system drivers include the economic, political, social, and technical influences that constrain, enable, or alter the characteristic of components in the system. Each system driver can have attributes that describe characteristics of each component. For a hospital, system drivers might include “government regulations,” “city utilities,” and “cost of electricity.”

System Drivers × System Drivers Interactions. System driver to system driver interactions may be important. For the hospital, an example of this type of interaction is the effect of “pharmaceutical pricing” (system driver) on “federal pharmaceutical subsidies” (system driver).

4.2. Stakeholders

The Stakeholder Matrix represents the social domain. Stakeholders are the human entities that contribute to the goals of the system and control components within the system. The

extent of their control of the system defines the system boundary. To identify the stakeholders for a system, it is useful to ask the four questions suggested by Maier and Rechtin [2000]: Who benefits? Who pays? Who provides? And who loses? Clarkson [1995] discusses stakeholders and stakeholder analysis.

Stakeholders × Stakeholders Interactions. Stakeholder-to-stakeholder interactions form the social network for a system. These can be analyzed in several ways [Wasserman and Faust, 1994]. In a hospital, social network analysis can be used to understand interactions between staff (stakeholder to stakeholder) to improve organizational design, information flow, and process efficiency.

4.3. Objectives

The Objectives Matrix represents the portion of the functional domain that defines the combined purposes of the system.

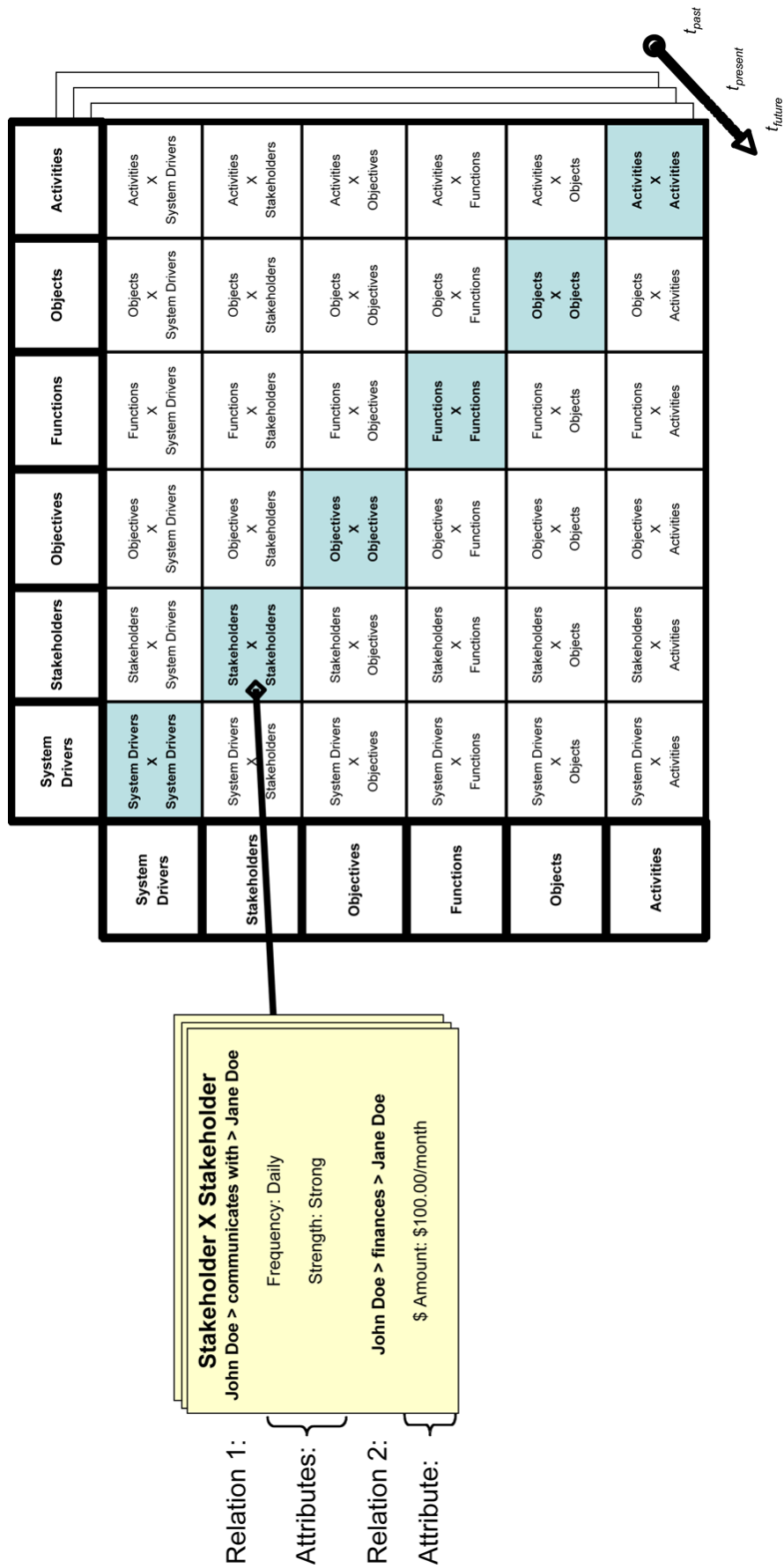


Figure 4. ES-MDM sample edges and attributes. [Color figure can be viewed in the online issue, which is available at wileyonlinelibrary.com.]

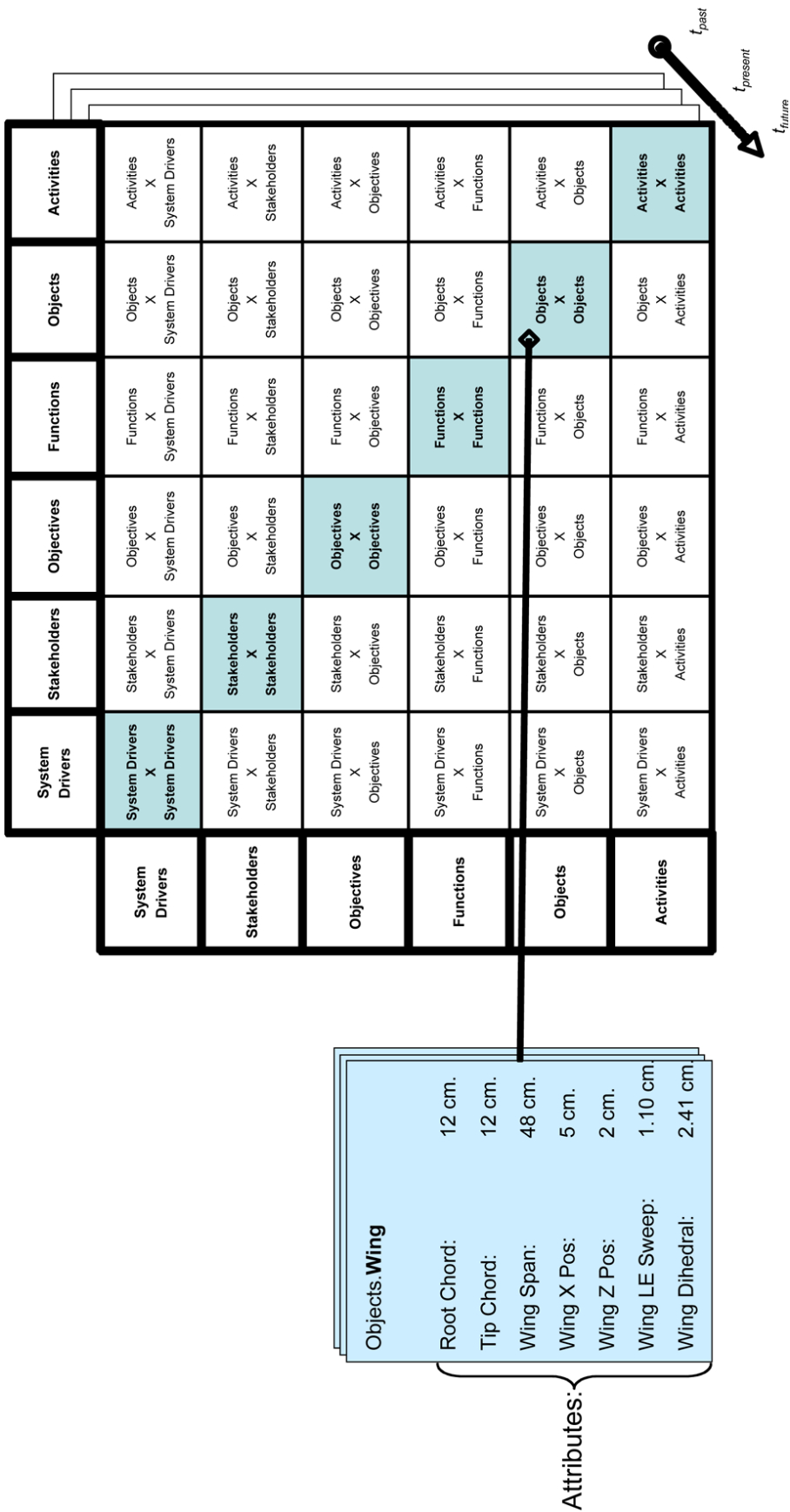


Figure 5. ES-MDM sample node and attributes. [Color figure can be viewed in the online issue, which is available at wileyonlinelibrary.com.]

Objectives include all articulated and unarticulated (implied) customers' needs for the system. A hospital objective might be: "To provide responsive emergency medical care." Attributes of objectives include quantifiable requirements or performance parameters, such as "average response time to medical emergencies."

Objectives × Objectives Interactions. The Objectives Matrix allows modelers to represent the interactions between objectives. For example, in a hospital the objectives "minimize operating costs" and "maximize employee satisfaction" may be opposing goals to be considered in planning.

4.4. Functions

Functions describe what the system does to achieve objectives. They represent the second part of the functional domain. All functions must relate to at least one objective either directly or through another function. They can have measurable attributes. They may also relate to the form of the systems. Each object and activity within the system maps to functions of the system. In a hospital a function that supports a system objective "to provide inpatient/outpatient health care" is the function "Provide Emergency Care."

Functions × Functions Interactions. Pahl and Beitz [1996] define the types of flows that exist between functions in products. These include signals (information), energy, material, and spatial relations. There can also be abstract relations between functions, such as hierarchic relations. In a hospital, the "Provide Emergency Care" function can be decomposed into subfunctions such as: "In-process Patients", "Retrieve Information", etc. See Blanchard and Fabrycky [2006] for a deeper discussion on functional analysis, decomposition, and allocation.

4.5. Objects

The Object Matrix represents the technical domain or the physical components of the system. These include infrastructure, objects needed to carry out the system functions, and physical entities used by the internal stakeholders. Other system objects include the architectural/physical entities required to carry out the functions or the physical "form" of the system. In the hospital, objects include software, hardware, and infrastructure.

Objects × Objects Interactions. Object-to-object interactions represent how the technical components interact. The Objects Matrix contains the information found in classic engineering models of technical systems. An example from a hospital is a computer network in which the "computer terminals" represent discrete objects and "signals flows" relate objects.

4.6. Activities

The Activities Matrix represents the process domain and includes the processes, procedures, tasks, and work units associated with an engineering system.

Activities × Activities Interactions. Activity-to-activity interactions have been well defined and studied [Steward, 1981; Warfield, 1990; Eppinger et al., 1994; Browning, 2009].

In engineering systems, the understanding of task-to-task dependency is essential when trying to understand certain system behaviors. In a product development, for example, the examination of task dependency can identify unnecessary iterations and feedback cycles that cause inefficiency. These dependencies can often be modeled with techniques ranging from process modeling to discrete event simulation. In a hospital, the activities associated with a particular medical procedure can be represented as parallel, sequential, and series tasks that represent all tasks from preoperation to postoperation.

4.7. Cross-Domain Interactions

This section examines and gives examples of important cross-domain interactions.

System Drivers × Stakeholders Interactions. System drivers often affect stakeholder components. For example, changing "demographics of a community" (system driver) requires that a hospital reorganize the mixture of specialties of "doctors" (stakeholders) employed by a hospital.

System Drivers × Objectives Interactions. System drivers often affect the system objectives. A housing development may increase the "community population" (system driver) that affects the "earning goals for the hospital" (objective).

System Drivers × Functions Interactions. A system driver affecting a function might include the hospital adding a new specialty of care (function), such as obstetrics, in response to "changing demographics" (system driver).

System Drivers × Objects Interactions. A system driver affecting an object might involve a "government regulation" (system drivers) that bans a "medical device" (object) used by the hospital.

System Drivers × Activities Interactions. System drivers affecting activities include "government regulation" (system driver) that requires special documentation or other actions (activities) after a medical procedure.

Stakeholders × System Drivers Interactions. Although stakeholders cannot control environmental factors, they can influence them. Thus "hospital doctors" (stakeholders) may volunteer in the community as hospital representatives to raise awareness for blood bank donations. The blood bank, an independent system that controls "community blood supply" (system driver), is an entity the hospital affects even though it does not control.

Stakeholders × Objectives Interactions. Stakeholder components define the objectives for a system. For example, if the hospital declares an initiative to reduce operating costs (objective), several stakeholders usually support, oppose, or are indifferent to it. An ES-MDM can store information regarding various stakeholder positions to inform a game theoretic model to analyze courses of action to align interests. See Dixit and Skeath [2003] for a tutorial on game theory and real world applications.

Stakeholders × Functions Interactions. The stakeholder-to-function matrix maps the influence of stakeholders to the corresponding functions. For example, the hospital's functions to "Provide Emergency Care" relates to the stakeholders who influence the function.

	System Drivers	Stakeholders	Objectives	Functions	Objects	Activities
System Drivers	The list and interactions of exogenous factors that act or acted on by the system	Relates the stakeholders that act on exogenous variables	Relates the objectives that act on exogenous variables	Relates the functions that act on exogenous variables	Relates the technical components that act on exogenous variables	Relates the activities that act on exogenous variables
Stakeholders	Relates the exogenous variables that act on system stakeholders	The list and interactions of the human entities within the system	Relates the objectives that act on stakeholders	Relates the functions that act on stakeholders	Relates the technical components that act on stakeholders	Relates the activities that act on stakeholders
Objectives	Relates the exogenous variables that act on system objectives	Relates the stakeholders that define or contribute to the system objectives	The list and interactions of combined purposes and goals of the system	Relates the functions that act on or relate to system objectives	Relates the technical components that act on system objectives	Relates the activities that act on system objectives
Functions	Relates the exogenous variables that act on system functions	Relates the stakeholders that act on system functions	Relates the objectives that are decomposed into system functions	The list and interactions of functions of the system	Relates the technical components that are traceable to system functions	Relates the activities that act on system functions
Objects	Relates the exogenous variables that act on system technical components	Relates the stakeholders that act on the technical components of the system	Relates the objectives that act on or constrain technical components	Relates the functions that are allocated to technical components	The list and interactions of technical components of the system	Relates the activities that act on technical components
Activities	Relates the exogenous variables that act on the system activities	Relates the stakeholders that engage in or act on the activities of the system	Relates the objectives that act on or constrain system activities	Relates the functions that are allocated to system activities	Relates the technical components that act on system activities	The list and interactions of activities of the system

Figure 6. Summary of ES-MDM domain interactions. [Color figure can be viewed in the online issue, which is available at wileyonlinelibrary.com.]

Stakeholders $\times$ Objects Interactions. Stakeholders in the system generally control or supervise the operation of the technical components. There are thus interactions between stakeholders and objects. In the hospital, a “radiology technician” (stakeholder) may oversee an “X-ray machine” (object).

Stakeholders $\times$ Activities Interactions. Human components contribute and support many tasks within the system. In a hospital, a “pharmacist” (stakeholder) “dispenses medicine” (activity) for a particular medical procedure.

Objectives $\times$ System Drivers Interactions. System objectives often affect exogenous variables. In a hospital, the failure to “respond to medical emergencies” (objective) affects “public perception” (system driver) of the quality of medical care that the hospital provides.

Objectives $\times$ Stakeholders Interactions. System objectives can affect stakeholders. Thus a hospital’s goal “to maximize profit” (objective) can affect the doctor salaries (stakeholder attributes).

Objectives $\times$ Functions Interactions. The functional decomposition of a system begins with its objectives of the system. For example, in the case of emergency response the goal of “providing responsive medical care” (objective) in-

cludes the functions: “receive message,” “transport patient,” and “treat patient.”

Functions $\times$ Objectives Interactions. Since all functions relate directly or through other functions to the system objectives, there will be several function-to-objective interactions. For example, in a resource allocation model, an analyst can measure the money spent per objective by summing the expenditures for the corresponding support functions.

Functions $\times$ Objects Interactions. Functional allocation maps function to the form of the system. Each function is likely to have a corresponding physical component. For example, the function “Store Information” corresponds to the hospital’s “medical records” (form).

Functions $\times$ Activities Interactions. Interactions exist between the systems’ activities performed and its functions. For example, the function “distribute medication” corresponds to the activities and procedures in the chain of tasks associated with delivering medicine to patients.

Objects $\times$ System Drivers Interactions. There are many interactions between the objects in an engineering system and the system drivers. In a hospital, there is a relationship between the its “water consumption” (object attribute) and the “city water supply” (system driver).

Objects × Functions Interactions. All objects in the system can be described with a functional description. In the case of the hospital, the cost attributes for the objects relate to those for the function. This relationship between objects and function costs has been well described in the value engineering literature [Fowler, 1990]. For example, the “costs” (object attribute) of a hospital’s information system relate to those of the hospital’s sub-function to “manage information.”

Objects × Activities Interactions. Most activities within an engineering system require objects in order to be accomplished. Medical procedures require medical devices. “Knee surgery” (activity) requires a “scalpel” (object).

Activities × System Drivers Interactions. Activities often interact with system drivers. For example, analyses of procedures (activities) performed by a hospital are used to inform “community health trends” (system driver).

Activities × Stakeholders Interactions. Feedback from activities is often of interest to particular stakeholders in an engineering system. For example, the “success rate of a particular procedure” (activity attribute) is reported to a “hospital administrator” (stakeholder).

Activities × Objects Interactions. An engineering system includes various activity-to-object interactions. In a hospital, “hospital internal blood supply” (object) is affected by the various surgical procedures (activities) being performed.

4.8. Time

The ES-MDM is designed to represent a system as it changes over time. It must be able to represent changes both in the arrangement of nodes and relationships, and in systems attributes. Capturing time within the ES-MDM is implemented through the system attributes for the nodes and relations. The ES-MDM declares an existence attribute that defines the time intervals each node or relation exists within the system. Once these are defined, the ES-MDM allows the modeler to define how particular attributes change over time. Attributes can be defined as discrete or continuous, time dependent functions.

In the case of a hospital, the “costs for particular pharmaceutical products” (object attributes) could be treated as variables that change quarterly. These discrete price changes are stored in the ES-MDM.

5. INSTRUCTIONS FOR BUILDING AN ES-MDM

The ES-MDM is useful tool for a variety of analysis including systems design, project management, and systems analysis. The construction of an ES-MDM is an iterative, nonlinear process as systems engineers have the flexibility to add, subtract, or change information about the system throughout the engineering process. Before building an ES-MDM, the analyst must determine the goal of the project, define the system to be modeled, and determine the sources of information and data. Depending on the goal of the analysis, the analyst may model some or all of the engineering systems domains.

For a systems design project, a systems engineer using the ES-MDM would begin by listing the stakeholders involved in the design to include customers, individuals involved in the

design process, and suppliers. This information is used to construct the Stakeholders Matrix.

The next matrix to be constructed is the Objectives Matrix. Sources of information would include requirements documentation, marketing information from customer interviews and focus groups, and other sources. While constructing the objectives matrix, systems engineers can relate each objective to the stakeholders in the Stakeholders × Objectives Matrix. A systems engineer can derive number of analytical insights from these three matrices that compactly represent the key stakeholders and their objectives. This information is vitally important throughout the system lifecycle since systems engineers should understand which stakeholders affect and are affected by changes in objectives.

Next, systems engineers can construct the Functions Matrix by decomposing the systems objectives into constituent functions. Functional allocation relates the Functions Matrix to the Objects Matrix (Functions × Objects Matrix) and Activities Matrix (Functional × Activities Matrix) as systems designers ensure proposed designs support the system functions.

The Objects Matrix represents the technical design. It relates technical subsystems and components to each other. The Activities Matrix relates the system processes and tasks to one another. The ES-MDM also allows systems engineers to relate stakeholders to activities (Stakeholders × Activities Matrix), objects to activities (Objects × Activities Matrix) to capture important information about connectivity between domains.

Analysts can identify, store, and relate exogenous factors to be considered in the Systems Drivers Matrix. This allows them to capture important information that is often overlooked. As the system design matures over its life-cycle, the ES-MDM can store system changes within and across domains.

For project management, a systems engineer might follow the same procedure in constructing an ES-MDM. Depending on the maturity of the system being analyzed, the sources and quality of information required to build the ES-MDM might vary.

Simple spreadsheet software (e.g., Microsoft Excel) can generally be used to construct a simple ES-MDM and offers a variety of valuable mathematical options. Steward [1981], Browning [2002], and Sharman and Yassine [2003] show how simple binary adjacency matrices can be used to improve systems designs. However, more sophisticated tools are required to realize the full potential of an ES-MDM to a systems engineering project.

6. COMPUTERIZED TOOL FOR BUILDING A FULLY ATTRIBUTED ES-MDM

A fully attributed ES-MDM is a highly complex representation of an engineering system. To simplify the model-building process, a team of MIT researchers developed software, the System Modeling and Representation Tool (SMaRT). Other knowledge-based engineering tools have been developed to store systems engineering data such as

ICAD, Dassault’s Knowledgeware, IBM’s Rational System Architect, and Vitech’s CORE.

What makes SMaRT unique is its ability to construct systems models, like the ES-MDM, from systems documentation of text documents. It does this by using a new procedure for model building called Qualitative Knowledge Construction (QKC). SMaRT stores the information for the different classes of nodes, relations, and attributes over time and allows for the easy retrieval of information and its export for mathematical analysis. For a brief discussion of SMaRT, see Bartolomei [2007].

Qualitative Knowledge Construction is a mode of incorporating qualitative information into systems models. Information can be collected through interviews, observation, and documentation. It is converted into text that is subsequently coded or “tagged” by the categories (nodes, relationships, attributes). Once coded, the data are automatically inserted into a rich data structure. The system is thus visually represented by the distribution of the values of the cells; the evidentiary data supporting that representation are contained within the cells. By developing a program for translating textual reports of observations and interviews into coded data capable of being systematically and quantitatively analyzed,

the researcher can translate qualitative information into quantifiable data.

In practical terms, QKC offers a means for better book-keeping when modeling an engineering system. It provides traceability of assumptions to data sources. Dong [2002] found that much of the information on an engineering system resides in the minds of the humans involved with it, not in technical documentation. QKC provides a means of capturing this knowledge and thus providing a transparency of assumptions and traceability to sources that rarely exists in the study or practice of engineering. By explicitly establishing this transparency and traceability, QKC removes epistemological barriers leading to better model building, design, and management.

7. USING THE ES-MDM

Building an ES-MDM to model a system provides many advantages. The ES-MDM provides a logical framework for organizing and representing information about a systems engineering project. Tools are available to take the information in the ES-MDM to produce classic systems engineering

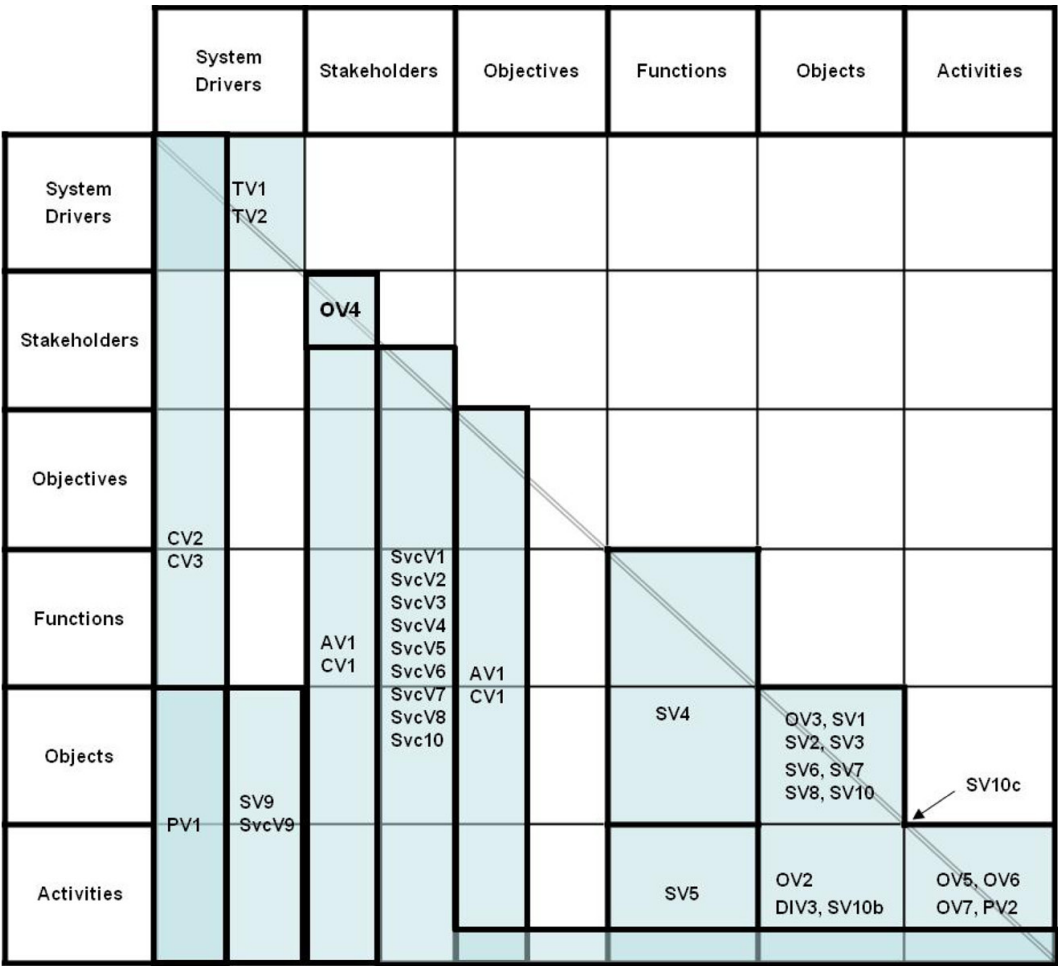


Figure 7. DoDAF views captured in ES-MDM. [Color figure can be viewed in the online issue, which is available at wileyonlinelibrary.com.]

products such as the DoDAF architecture viewpoints (i.e., the OV, SV and TV). Figure 7 represents where DoDAF information can be represented in the ES-MDM.

The ES-MDM can also be used for quantitative system analysis. Algorithms have been developed to analyze particular domains within the system. Sharman and Yassine [2003] present techniques for analyzing product architectures that can be applied to the Objects Matrix in the ES-MDM. Browning [2002] presents algorithms to analyze systems processes using the DSM that can be applied to the Activities Matrix in the ES-MDM. Bustnay and Ben-Asher [2005] present a system partitioning method that can be applied to an ES-MDM using graph theory on N^2 charts to identify independent subsystems to simplify system definition, improve interface and module development, and improve system integration and verification. Guenov and Barker [2005] present the COPE design-decomposition model that relates information within and across functional, physical, and process domains. By applying COPE to an ES-MDM, analysts can better identify potential conflicts in the system design and better understand the effect of design changes. Maurer and Lindemann [2008] present graph theoretic algorithms using multiple-domain

matrices to analyze engineered systems. These algorithms can be applied to matrices within the ES-MDM.

Bartolomei [2007] used such methods to improve a systems design for the US Air Force Research Laboratory (AFRL). AFRL is the US Air Force's leading organizations dedicated to the discovery, development, and integration of war fighting technologies for air, space, and cyberspace forces. AFRL was tasked to develop a miniaturized unmanned air vehicle (MAV) for US Special Forces. This project integrated several technologies in development within AFRL, industry, and academia. Over a 2-year period, Bartolomei observed this project and constructed an ES-MDM that captured a time-series data set. He used it to analyze the dynamic complexity of the product development, in particular the impact of engineer turnover within the design organization, the effects of changing requirements on the design, and the evolution of the UAV system technical design. This analysis identified platform and modularity opportunities in the design by allowing system engineers to explore the sources and effects of design changes on the product development.

An ES-MDM of the MAV product development system was constructed using the QKC methodology. Figure 8 is a

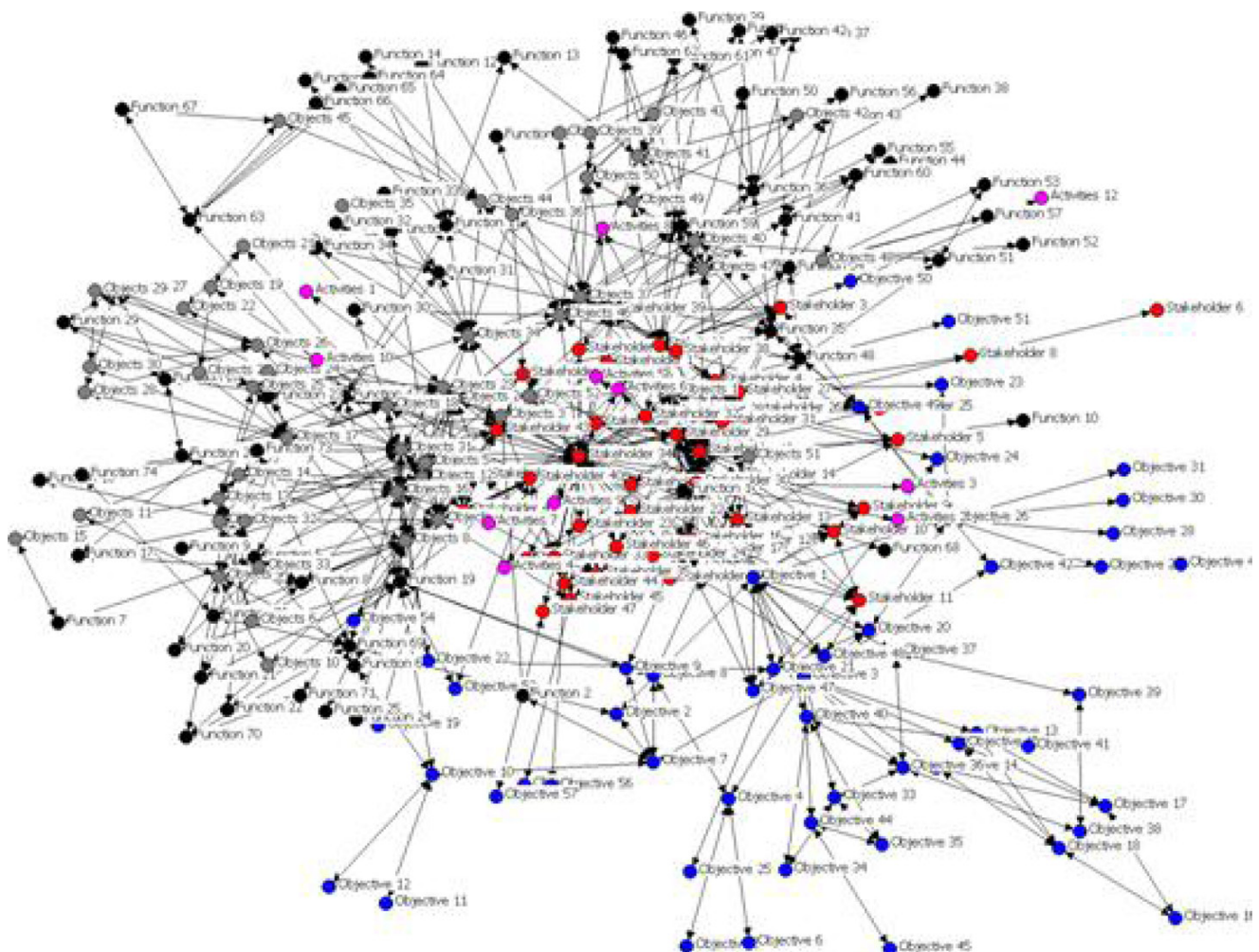


Figure 8. A network representation of ES-MDM. [Color figure can be viewed in the online issue, which is available at wileyonlinelibrary.com.]

Rank	Objects Network Alone		Rank	MAV-PD	
1	Engine Subsystem	372.797	1	Autopilot Subsystem	1977.195
2	Ground Station Transmitter	271.913	2	Communication Subsystem (Datalink)	1822.32
3	Control Subsystem	243.768	3	Ground Station Subsystem	1749.317
4	Ground Station Subsystem	211.846	4	Air Vehicle	1388.325
5	Ground Station Software	197.242	5	Wing Subsystem	1298.756
6	Actuator #1	153.585	6	Battery Subsystem	1013.186
7	Wing Subsystem	138.008	7	Fuselage Subsystem	1007.738
8	Battery Connectors	134.774	8	Ground Station Software	992.118
9	Ribs	127.143	9	Control Subsystem	967.44
10	Wing Composite Structure	102.837	10	Fuselage Structure	967.42

Figure 9. Betweenness centrality for objects.

network representation of the ES-MDM early in the system development. The colors of the nodes represent the different domains. Treating the ES-MDM data as network allows us to analytically examine system characteristics using methods including network theory and graph theory. The analysis aimed at getting insight into important questions, including: Is our team well organized for the task? Are we allocating resources appropriately? Do we have the right processes in place to execute the project?

An important question integrating some of these concerns is “are we managing this project at the subsystem or component level?” Insights into this question drive organizational structure, resource decisions, and process development. To glean insight into this question, we used a common network metric, betweenness centrality. This measures the number of times an element within the ES-MDM occurs on the short path connecting two system elements. We used it to compare the top ten ranked objects in the Objects Matrix with the top ten ranked objects in the entire ES-MDM.

The results shown in Figure 9 indicated subsystems have greater connectivity than components in the larger system. From a management perspective, this suggested that the MAV program was managed at the subsystem level rather than the component level.

Next, network analysis was used to identify and examine critical system nodes (people, technology, processes). We analyzed the connectedness of stakeholders within the MAV project to duplicate the previous findings. In social network analysis, betweenness centrality is associated with the control

of information: Stakeholders with higher betweenness have greater influence on a social network than those with lower betweenness.

Figure 10 shows the results of the analysis: the top ten stakeholders ranked by betweenness measure within the social network compared to the entire MAV ES-MDM at time 3. Each acronym “XXXX” represents a particular individual.

In the social domain, the highest ranked stakeholders were the chief engineer “PMWJ” and USAF customer “STCC.” PMWJ was the chief engineer; STCC the customer’s representative; PMBI the program manager; SPOMD, SPOKE, and SPOGR were USAF staff responsible for managing the contract; and KTRDM the lead contractor responsible for the ground station and production contract. These results were as expected.

The rankings changed however when the analysis was expanded to include the functional, technical, and process domains, as shown on the right. For example, subcontractor KTRDM’s ranking surpassed that of the program manager PMBI. The ES-MDM thus revealed that KTRDM, a stakeholder seemingly less important when looking at the social network, was far more “connected” and had greater influence in the overall MAV engineering system. The ES-MDM provided a means to better understand the roles of stakeholders involved in the system.

The ES-MDM was used to examine how stakeholder characteristics changed over time. Figure 11 compares stakeholder’s PMWJ and STCC’s degree centrality (the count of connections for a node) and betweenness over a several peri-

Rank	MAV-PD Social Network	Betweenness	Rank	MAV-PD Entire Network	Betweenness
1	PMWJ	500.199	1	PMWJ	10972.993
2	STCC	199.471	2	KTRDM	3680.017
3	PMBI (MAV-PD PM 3)	84.154	3	KTRNM	1972.081
4	SPOMD	54.547	4	STCC	1556.707
5	SPOKE	45.143	5	PMBI (MAV-PD PM 3)	1372.588
6	SPOGR	43.867	6	KTRRC	1004.062
7	KTRDM	40.153	7	KTRTT	618.312
8	STYA	21.676	8	KTRBR	390.463
9	STSP	20.47	9	SPOMD	293.354
10	PMFC	15.23	10	STYA	275.212

Figure 10. Betweenness centrality for stakeholders.

					PM and ST Replacements	
PMWJ		Time 1	Time 2	Time 3	Time 4	Time 5
Degree Centrality		38	46	61	53	8
Betweenness		3643	5427	11836	10331	1769,191
STCC						
Degree Centrality		16	21	28	23	4
Betweenness		820	866	1667	3501	240,846
MAV-PD Avgs						
Degree Centrality		4.87	4.93	4.7	4.96	3.777
Betweenness		238	258	280	296	338,777

Figure 11. Centrality measures for MAV stakeholders over time.

ods. The analysis compared these centrality measures with those for their replacements at time 5. The metrics for both PMWJ and STCC grew over time and were always much larger compared to those of the average individual within the MAV system. At time 5, however, both PMWJ and STCC were removed and replaced with two new agents. The numbers show that the replacements had significantly smaller centrality measures, an indication that they were not as well connected in the system. In this case, the ES-MDM and analysis can be used as a tool to mitigate downside risks associated with personnel changes in a system. For the MAV program, the changeover in personnel correlated with a decline in cost, schedule, and technical performance. A benefit of the ES-MDM is that it gives new personnel a means to engage more deliberately in the system by revealing important connections within and across domains.

The next example shows how network measures provide quantitative insights to the entire MAV development system over time. Newman [2003] performed network analysis for several technological systems. These calculations are listed in Figure 12. As shown at the bottom five rows in Figure 12, the size of the MAV network and the density of the relationships changed over five different periods. These changes are not surprising, as frequent organizational and technology changes were well documented. It is interesting to note the difference in network metrics at time 5. The degradation of the number of relations far exceeds that of the number of nodes. The average degree $\langle k \rangle$ and clustering coefficient is lower compared to those in other periods and path length seems much longer. This observation prompted a reexamination of the system for insights as to what might have happened.

At time 5, the changes in the network metrics reflect the changes within the system. The loss of PMWJ and STCC was

Network	Type	# of nodes <i>n</i>	# or relations <i>m</i>	avg. degree <i><k></i>	avg. path length <i>l</i>	Rand Grph	Clus. Coef <i>C</i> ⁽¹⁾	Clus. Coef <i>C</i> ⁽²⁾	Rand Grph	
						theo I			theo C	
						$\frac{\log n}{\log <k>}$			<i><k>/n</i>	
Technological										
Internet	undirected	10 697	31 992	5.98	3.31	5.19	0.035	0.39	0.0005	
power grid	undirected	4 941	6 594	2.67	18.99	8.67	0.1	0.08	0.0004	
Train routes	undirected	587	19 603	66.79	2.16	1.52		0.69	0.12	
software packages	directed	1 439	1 723	1.2	2.42	39.81	0.07	0.082	0.0008	
software classes	directed	1 377	2 213	1.61	1.51	15.18	0.033	0.012	0.0012	
electronic circuits	undirected	24 097	53 248	4.34	11.05	6.87	0.01	0.03	0.0002	
peer-to-peer network	undirected	880	1 296	1.47	4.28	17.6	0.012	0.011	0.002	
BOSTON T	undirected	81	3684	45.48	1.887	1.15	0.94	0.857	0.565	
Biological										
Silwood Park Food Web		154		4.75	3.4	3.23266053	0.15	0.03	0.03084416	
C. Elegans		282		14	2.65	2.13784938	0.28	0.05	0.04964539	
Social Networks										
Movie Actors	undirected	225226		61	3.65	2.99811184	0.79		0.00027084	
LANL Coauthorship	undirected	52909		9.7	5.9	4.78685129	0.43		0.00018333	
MAV-PD Time 1	undirected	184	884	4.87	3.61	3.2941417	0.293	0.209	0.02646739	
MAV-PD Time 2	undirected	196	956	4.93	3.665	3.30845964	0.314	0.2	0.02515306	
MAV-PD Time 3	undirected	262	1232	4.702	3.621	3.59714977	0.316	0.196	0.01794656	
MAV-PD Time 4	undirected	223	1106	4.96	3.667	3.37651579	0.299	0.19	0.02224215	
MAV-PD Time 5	undirected	206	778	3.77	4.437	4.01475136	0.221	0.126	0.01830097	

Figure 12. MAV network metrics.

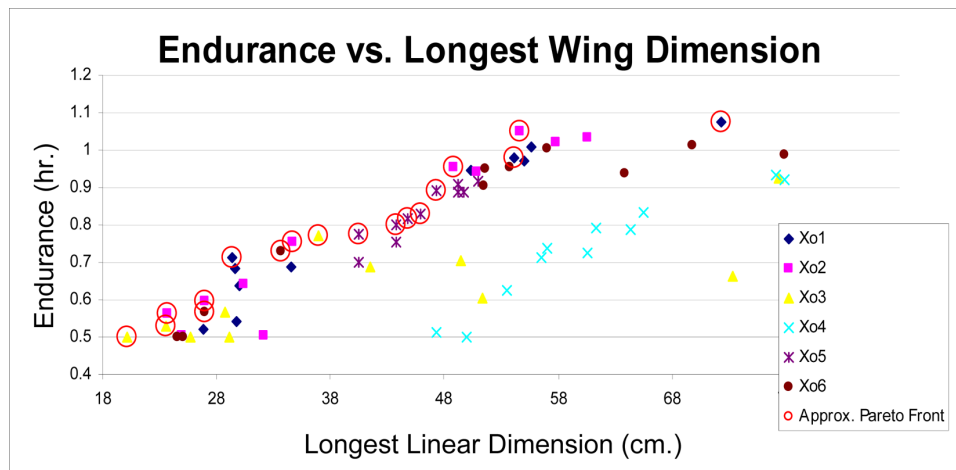


Figure 15. MAV design trade-space. [Color figure can be viewed in the online issue, which is available at wileyonlinelibrary.com.]

directed. [Eckert and Clarkson, 2004]. Emergent change is unexpected and occurs from within the boundary of the system. During the development of the MAV, one of the engineers on the team accidentally broke the wing of a prototype, while “playing” with system at his desk. This failure propagated throughout the system, causing schedule delays, work disruptions, and other problems. In contrast, directed change is prescribed by stakeholders, usually as new requirements. In the previous example, while bending the wing the engineer conceived of a new “wing roll-up” design for the system. He called the contractor responsible for the wing and levied the new requirement. This directed change initiated a completely new wing design and affected various other aspects of the system.

The ES-MDM was used to brainstorm various scenarios and examine how the MAV would need to change due to new technologies, new threats, or other factors. One change scenario involved new stakeholders (i.e., the Army) who requested a MAV with longer endurance. Another scenario assumed suppliers would soon deliver a suite of miniaturized components (batteries) with better technical performance. The team used the Excel model for the MDO analysis to identify which system variables most significantly affected the objectives. A cost estimate was performed to determine the costs of different changes.

The wing and tail connectors and the batteries subsystems were identified as opportunities for flexibility in the system. A redesign of the fuselage and the battery interfaces could enable the MAV to accommodate more batteries and larger wings if new requirements demanded these features.

In short, the ES-MDM holds promise as a tool to gather, organize, and analyze information to better understand the structure and behavior of an engineering system. By using QKC to construct an ES-MDM, researchers were able to gather information about a complex system with transparency. All data in the ES-MDM was traceable to primary sources and open to discourse and scrutiny. The data structure allowed researchers to relate and organize data that included

both the technological and the often underrepresented social domain.

The ES-MDM also holds promise as an industrial tool for systems engineers. The information used to build common systems level-models and products such as QFD, House of Quality, DoDAF, and DSM can be captured in a fully attributed ES-MDM. Plus, the ES-MDM captures information about the social and environmental domains important to systems engineering projects. Lastly, the ES-MDM has the added advantage of storing time-series information about project.

8. FUTURE WORK

The ES-MDM has been applied to a number of engineering projects. While its conceptual model has proven sufficient to describe the systems of interest, we do not know if the five domains constitute a complete set. In particular, the environment domain needs to be more defined.

Ongoing research has been exploring interactions between two or more engineering systems. This uses multiple ES-MDM's to represent distinct and interacting systems where each exists in the other's environmental domain. Its results show promise as means to explore the management and integration of disparate systems.

The underlying conceptual model and the ES-MDM can improve existing systems engineering software tools (e.g., Vitech's CORE) as well as systems modeling frameworks (i.e., DoDAF 2.0). They provide a broader model for representing engineering systems. Commercially available systems-modeling tools could also integrate analytical methods.

Lastly, there is much to be done to develop guidelines for implementation. Better strategies for data abstraction (how much data to too much, too little?), and the development of criteria for when and how systems models should be applied are worthwhile subjects for research.

9. SUMMARY

The ES-MDM allows modelers to represent each of the Engineering Systems domains. They can map interactions within and across domains and to parameterize these relations. They can also represent the system as it changes over time, both in the structure (changing nodes and relations) and attributes of the components.

Finally, the ES-MDM allows us to treat social and technical interactions on an equivalent basis for the first time. This enables us to construct social and technical networks and to study their interactions with network analysis tools.

ACKNOWLEDGMENTS

Jason Bartolomei thanks Thomas Hughes for his comments and feedback; David Broniatowski, Matt Richards, Adam Ross, Nirav Shah, and Jennifer Wilds at the MIT Systems Engineering Advancement Research Initiative (SEARI); Chris Glazner, Chris Lawson, Chris Magee, Joel Moses, Joe Sussman, and Dan Whitney in MIT's Engineering Systems Division (ESD); and Tyson Browning.

REFERENCES

- L.R. Ackoff, Science in the systems age: Beyond IE, OR, and MS, *Oper Res* 21(3) (1973), 661–671.
- G.T. Allison and P. Zelikow, *Essence of decision: Explaining the Cuban missile crisis*, Longman, New York, 1999.
- R.J. Badham, C.W. Clegg, and T.D. Wall, *Sociotechnical theory*. In W. Karwowski (Ed.), *International Encyclopedia of Ergonomics and Human Factors*, Taylor and Francis, New York, 2000.
- J.E. Bartolomei, Qualitative knowledge construction for engineering systems: Extending the design structure matrix methodology in scope and procedure, PhD thesis, Massachusetts Institute of Technology, Engineering Systems Division, Cambridge, MA, 2007.
- J.E. Bartolomei, M. Cokus, J. Dahlgren, R. de Neufville, D. Maldonado, and J. Wilds, Analysis and applications of design structure matrix, domain mapping matrix, and engineering system matrix frameworks, Massachusetts Institute of Technology, Cambridge, MA, 2007.
- L.P. Beckerman, The application of complex systems science to systems engineering, *Syst Eng* 3 (2000), 96–102.
- S. Beer, *Cybernetics and management*, English Universities Press, London, 1967.
- L.v. Bertalanffy, *General system theory: foundations, development, applications*, G. Braziller, New York, 1968.
- B.S. Blanchard and W.J. Fabrycky, *Systems engineering and analysis*, 4th edition, Prentice Hall, Upper Saddle River, NJ, 2006.
- K.E. Boulding, General systems theory: The skeleton of science, *Management Sci* 2 (1956), 197–208.
- J.H. Brill, Systems engineering—a retrospective view, *Syst Eng* 2 (1999), 258–266.
- T.R. Browning, Applying the design structure matrix to system decomposition and integration problems: A review and new directions, *IEEE Trans Eng Management* 48(3) (2001), 292–306.
- T.R. Browning, Process integration using the design structure matrix, *Syst Eng* 5(3) (2002), 180–193.
- T.R. Browning, The many views of process: Toward a process architecture framework for product development processes, *Syst Eng* 12 (2009), 69–90.
- D.M. Buede, *The engineering design of systems: Models and methods*, Wiley, New York, 2000.
- M. Bunge, *Ontology II: A world of systems*, D. Reidel, Dordrecht, 1979.
- T. Bustnay and J.Z. Ben-Asher, How many systems are there? Using the N^2 method for systems partitioning, *Syst Eng* 8 (2005), 109–118.
- M. Callon, “Society in the making: The study of technology as a tool for sociological analysis,” *The social construction of technological systems*, W.E. Bijker, T.P. Hughes, and T.J. Pinch (Editors), The MIT Press, Cambridge, MA, 1990, pp. 83–103.
- CESUN, <http://www.cesun.org/>, Cambridge MA, 2010.
- M.B.E. Clarkson, A stakeholder framework for analyzing and evaluating corporate social performance, *Acad Management Rev* 20(1) (1995), 92–117.
- C.A. Cooper, M.L. Ewoldt, S. A. Meyer, and E. W. Talley, A systems architectural model for man-packable/operable intelligence, surveillance, and reconnaissance mini/micro aerial vehicles, Air Force Institute of Technology, Department of Aeronautical Engineering, Wright-Patterson Air Force Base, OH, 2002.
- M. Danilovic and T.R. Browning, Managing complex product development projects with design structure matrices and domain mapping matrices, *Int J Management* 25 (2007), 300–314.
- R. de Neufville, *Architecting/designing engineering systems using real options*, ESD Intern Symp, MIT, Engineering Systems Division, Cambridge, MA, 2002.
- A. Dixit and S. Skeath, *Games of strategy*, Norton, New York, 2003.
- DoD, DoD architecture framework: Version 1.0, D. A. F. W. Group, Department of Defense, Washington, DC, 2003.
- DoD, DoD architecture framework: Version 2.0, D. A. F. W. Group, Department of Defense, Washington, DC, 2009.
- R.J. Dodder, J. McConnell, A. Mostashari, and J. Sussman, The concept for using the “CLIOS Process”: Integrating the study of physical and policy systems using Mexico City as an example, MIT Engineering Systems Working Papers, Massachusetts Institute of Technology, Cambridge, MA, 2005.
- Q. Dong, Predicting and managing system interactions at early phase of the product development process, PhD thesis, Massachusetts Institute of Technology, Mechanical Engineering, Cambridge, MA, 2002.
- C. Eckert, P.J. Clarkson, and W. Zanker, Change and customisation in complex engineering domains, *Res Eng Des* 15 (2004), 1–21.
- F. Emery, “Characteristics of socio-technical systems,” *The social engagement of social science*, Volume II: The socio-technical perspective, E. L. Trist and H. Murray (Editors), University of Pennsylvania Press, Philadelphia, 1993.
- S.D. Eppinger, Patterns of product development interactions, ESD Intern Symp, MIT Engineering Systems Division, Cambridge MA, 2003.
- S.D. Eppinger, D.E. Whitney, R.P. Smith, and D.A. Gebala, A model-based method for organizing tasks in product development, *Res Eng Des* 6 (1994), 1–13.
- ESD, Engineering systems strategic plan, Massachusetts Institute of Technology, Engineering Systems Division, Cambridge MA, 2008, http://esd.mit.edu/HeadLine/ESD_StrategicPlan2008.pdf.
- J.W. Forrester, *Industrial dynamics*, Productivity Press, Cambridge, MA, 1961.

- J.W. Forrester, *Urban dynamics*, Productivity Press, Cambridge MA, 1969.
- T.C. Fowler, *Value analysis in design*, Van Nostrand, New York, 1990.
- M.R. Genesereth and N.J. Nilsson, *Logical foundations of artificial intelligence*, Morgan Kaufmann, Los Altos, CA, 1984.
- T.E. Graedel and B.R. Allenby, *Industrial ecology*, Prentice Hall, Upper Saddle River, NJ, 2003.
- M.D. Guenov and S.G. Barker, Application of axiomatic design and design structure matrix to the decomposition of engineering systems, *Syst Eng* 8(1) (2005), 29–40.
- L.R. Guinta and N.C. Praizler, *The QFD book: The team approach to solving problems and satisfying customers through quality functional deployment*, AMACOM, New York, 1993.
- T.P. Hughes, *Networks of power: Electrification in Western society, 1880–1930*, Johns Hopkins University Press, Baltimore, MD, 1983.
- T.P. Hughes, “The evolution of large technological systems,” *The social construction of technological systems*, W.E. Bijker, T.P. Hughes, and T.J. Pinch (Editors), The MIT Press, Cambridge, MA, 1987.
- T.P. Hughes, *American genesis: A century of invention and technological enthusiasm, 1870–1970*, Penguin Books, New York, 1990.
- T.P. Hughes, *Rescuing Prometheus*, Pantheon Books, New York, 1998.
- H. Kitano, *Systems biology: A brief overview*, *Science* 295 (2002), 1662–1664.
- E. Laszlo, *The systems view of the world; the natural philosophy of the new developments in the sciences*, G. Braziller, New York, 1972.
- P.K. M’Pherson, A perspective on systems science and systems philosophy, *Futures* 6 (1974), 219–239.
- M.W. Maier and E. Rechtin, *The Art of Systems Architecting*, Second Edition, June, 2000.
- J.H. Marchal, On the concept of a system, *Phil Sci* 42 (1975), 448–468.
- M. Maurer and U. Lindeman, The application of the multiple domain matrix, *Proc IEEE Int Conf Syst Man Cybernet (SMC)* 2008.
- R.K. Merton, *Social theory and social structure*, The Free Press, New York, 1957.
- J.G. Miller, *Living systems*, McGraw-Hill, New York, 1978.
- J. Moses, *Foundational issues in engineering systems: A framing paper*, Engineering Systems Monograph, Massachusetts Institute of Technology, Cambridge, MA, 2004, <http://esd.mit.edu/resources/symposium2004.html>.
- M. Newman, The structure and function of complex networks, *SIAM Rev* 42(2) (2003), 167–256.
- G. Pahl and W. Beitz, *Engineering design: A systematic approach*, Springer, New York, 1996.
- T. Parsons, Evolutionary universals in society, *Amer Sociol Rev* 29 (1964), 339–357.
- E.C. Pielou, *An introduction to mathematical ecology*, Wiley-Interscience, New York, 1969.
- K. Popper, *The poverty of historicism*, Routledge, London, 1961.
- K. Popper, *Objective knowledge*, Oxford University Press, Oxford, 1972.
- E.S. Quade and W.I. Boucher, *Systems analysis and policy planning; applications in defense*, American Elsevier, New York, 1968.
- N. Rescher, *Cognitive systematization: A systems-theoretic approach to a coherentist theory of knowledge*, Rowan and Littlefield, Totowa, NJ, 1979.
- M.G. Richards, N. B. Shah, D.H. Rhodes, and D.E. Hastings, *Managing complexity with the Department of Defense Architecture Framework: Development of a system architecture model*, CSER: Conference on Systems Engineering Research, Los Angeles, CA, 2006.
- W. Rouse, Complex engineered, organizational and natural systems, *Syst Eng* 10(3) (2007), 260–271.
- A.P. Sage, *Systems engineering*, Wiley, New York, 1992.
- A.P. Sage and J. E. Armstrong, *Introduction to systems engineering*, Wiley, New York, 2000.
- P.M. Senge, *The fifth discipline: the art and practice of the learning organization*, Doubleday/Currency, New York, 2006.
- D.M. Sharman and A.A. Yassine, Characterizing complex product architectures, *Syst Eng* 7 (2003), 35–60.
- H.A. Simon, The architecture of complexity, *Proc Amer Phil Soc* 106 (1962), 467–482.
- N. Sproles, The difficult problem of establishing measures of effectiveness for command and control: A systems engineering perspective, *Syst Eng* 4 (2001), 145–155.
- D.V. Steward, The design structure system: A method for managing the design of complex systems, *IEEE Trans Eng Management* 28 (1981), 71–74.
- N.P. Suh, Axiomatic design theory for systems, *Res Eng Des* 10 (1998), 189–209.
- J. Sussman, *Collected views on complexity in systems*, ESD Symp Eng Syst Res Practice, Cambridge, MA, 2002, <http://esd.mit.edu/resources/symposium2004.html>.
- E.L. Trist, Some observations on the machine face as a socio-technical system, Document 341, Tavistock Institute, London, 1953.
- E.L. Trist and K.W. Bamforth, Some social and psychological consequences of the longwall method of coal getting, *Hum Rel* 4 (1951), 3–38.
- G. Vickers, *The art of judgement: A study of policy making*, Harper & Row, London, 1983.
- J.N. Warfield, *A science of generic design: managing complexity through systems design*, Intersystems Publications, Salinas, CA, 1990.
- J.N. Warfield and J. D. Hill, Unified program planning, *IEEE Trans Syst Man Cybernet Part A SMC*-2(5) (1972), 610–621.
- S. Wasserman and K. Faust, *Social network analysis: Methods and applications*, Cambridge University Press, New York, 1994.
- N. Wiener, *Cybernetics*, *Sci Amer* (1948), 14–19.



Jason E. Bartolomei is an active duty Air Force officer with experience in acquisitions, counterterrorism operations, and intelligence analysis. He has served as an engineer and program manager at the F-22 System Program Office, served as an Assistant Professor of Engineering Mechanics and Director of Systems Engineering at the US Air Force Academy, served as 2010 DARPA Service Chief's Fellow, and led the Joint Warfare Analysis Center's Counter Terrorism Support Team—where his team provided interdisciplinary analysis to US military leadership in support of worldwide, counterterrorism operations. He holds a PhD from MIT and is a research affiliate at MIT's Engineering Systems Division.



Daniel E. Hastings is Professor of Engineering Systems and Aeronautics and Astronautics and Dean for Undergraduate Education at MIT. He is a Fellow of the AIAA and a member of the International Academy of Astronautics. He is serving as a member of the National Science Board and the Applied Physics Lab Science and Technology Advisory Panel, as well as the chair of the Air Force Scientific Advisory Board. He is a member of the MIT Lincoln Laboratory Advisory Committee and is on the Board of Trustees of the Aerospace Corporation. He has served on several national committees on issues in National Security Space. Dr. Hastings was elected as a Fellow of INCOSE (the International Council on System Engineering) in June 2007.



Richard de Neufville is Professor of Engineering Systems at MIT. His teaching, research, and writing focus on the Design of Engineering Systems, with particular emphasis on Flexibility in Design. He was the founding chairman of the MIT Technology and Policy Program. He holds concurrent appointments at the University of Cambridge, the Instituto Superior Técnico of Portugal, and Harvard University. He was in the first class of White House Fellows, and assigned to Secretary of Defense McNamara.



Donna H. Rhodes is a Senior Lecturer and Principal Research Scientist in the MIT Engineering Systems Division, and Codirector of the Systems Engineering Advancement Research Initiative (SEARI). Her areas of specialization include technical and management theory and practices for architecting and design of complex systems, systems-of-systems, and enterprises. Prior to joining MIT, she had 20 years of experience in the aerospace, defense systems, systems integration, and commercial product industries. She is a Past President and Fellow of INCOSE, and is a recipient of the INCOSE Founders Award and several INCOSE Distinguished Service Awards. She has published numerous papers and research reports in the field of systems, and has coauthored industry and corporate engineering policies, standards, and technical reports.